Advanced Web Development — Assignment Brief

Module: CC-Y2-AWD — Advanced Web Development
Assessment: 100% continuous assessment, in two assessment points

Assessment point	Weighting	Code + screencast due	Viva
Assessment Point 1 — Work to date	30%	23:59, Friday 30 October 2026	Tuesday 3 November 2026
Assessment Point 2 — Final submission	70%	23:59, Friday 11 December 2026	Tuesday 15 December 2026

Module aim and learning outcomes

The aim of this module is to:

- improve your knowledge and skills in relation to the development and testing of web applications
- provide you with the understanding, skills and competencies to use server-side web application frameworks.

On successful completion of this module, you will be able to:

- **LO1** — Explain the principles of server-side Web Application Frameworks
- **LO2** — Design, develop, test and debug a web application using a server-side web application framework
- **LO3** — Document, explain and defend the design and implementation of web applications

---

The assignment

You will design and build a **Laravel web application for a case study of your own choice**, working on it across the whole 12 weeks of the module.

The **Book Reviews** application and its tutorial notes are your **worked example**. They show you, step by step, how to build each part of a Laravel application. You do **not** submit Book Reviews. You apply each technique from the tutorial to your own case study.

Your case study

Your case study must have:

- **at least one one-to-many (1:M) relationship** — e.g. one book has many reviews
- **at least one many-to-many (M:N) relationship** — e.g. a book has many authors, and an author has written many books

- **a main entity** — the "thing" your application is about (the equivalent of a Book in Book Reviews)
- **at least two roles**, and some pages that can only be accessed by users with a given role.

This is how Book Reviews maps onto those requirements:

Requirement	Book Reviews
Main entity	Book
1:M relationship	A Book has many Reviews
M:N relationship	A Book has many Authors, and an Author has many Books (`author_book` join table)
Roles	Only admins can create, edit and delete Books and Authors. Users can only edit and delete their own Reviews. (This will be added to the Book Reviews tutorial at a later date.)

Extensions

Once your core application is working, you can earn further marks at Assessment Point 2 by adding **extensions** — features that go beyond what the Book Reviews tutorial covers. Examples:

- image uploads with resizing or validation
- pagination and sorting on index pages
- an admin dashboard showing summary statistics
- a further relationship or entity
- email notifications

Discuss any extension with your lecturer before you build it, so that you both agree it is feasible and worth doing. Finish your core requirements first — an extension will not make up for missing core functionality.

Technical requirements

Your application must:

- use **Laravel 12** with **Blade** views (no Livewire, Inertia, React or Vue)
- use **Laravel Breeze** (Blade stack) for registration and login
- use **MySQL**, running in the **Docker environment** provided with Book Reviews
- use **Eloquent** models and relationships for database access (no raw SQL)
- use **migrations**, **factories** and **seeders**, so that `php artisan migrate:fresh --seed` builds your database with realistic demo data

- use **resource controllers** and **route model binding**
- validate all form input using Laravel's validation
- have **PHPUnit** tests that pass with `php artisan test`.

Using AI tools

You **may** use AI tools to help you **understand error messages and fix bugs**. We recommend that you **do not** use AI tools to write your code for you — writing the code yourself is how you learn.

If you do use AI to write code, you must be able to explain every line of it. **The viva will test this** (see below). Code you cannot explain will not earn marks, whoever wrote it.

Record your AI use as you go — you will need it for the *AI Agent Use* section of your design document at Assessment Point 2.

Read the Institute's guidance on AI and academic integrity before you start: https://iadt.libguides.com/AI/AcademicIntegrity

How you will be assessed

Your marks are based on:

1. **Your code**, submitted to your **classroom50.org** repository. Only your own case study should be in this repository.
2. **Your git commit history.** Commit and push regularly, with clear commit messages. A history that shows steady progress across the weeks is strong evidence of your own work. A history with one or two large commits near the deadline is not.
3. **A recorded screencast** of your application, uploaded to the OneDrive folder created for you.
4. **Your attendance and observed progress in class.**
5. **A viva** (a short one-to-one conversation with your lecturer about your code), which **gates** your marks — see below.

The viva

Vivas take place during scheduled class hours. A schedule with your time slot will be published in advance.

At each viva you will be asked to **trace one or two requests through your application, from start to finish**. For example: "A logged-in user submits the edit form for one of your main entity's records. Walk me through what happens." You will describe each point in your code that the request passes through and what it does there — for example:

> the route → any middleware → the controller action → validation → the model and its relationships → the database → the redirect or view that is returned

You will have your code open in front of you. You are not expected to have it memorised. You **are** expected to understand it.

The viva gates your marks. The viva confirms that the submitted work is your own and that you understand it:

Viva outcome	Effect on your mark
Confirmed — you can trace requests through your code and explain it	Your rubric marks stand.
Partly confirmed — you can explain some of your code but not all of it	Marks for criteria that depend on the code you could not explain are capped at 40%.
Not confirmed, or you do not attend	Your assessment point is graded as a **Fail**.

If you cannot attend your viva for a reason covered by the Institute's extenuating circumstances procedures, contact your lecturer as soon as possible so it can be rescheduled.

Assessment Point 1 — Work to date (30%)

Code and screencast due: 23:59, Friday 30 October 2026
Viva: Tuesday 3 November 2026

What we expect

By this point we expect you to have reached the equivalent of **Lesson 3** of the Book Reviews tutorial, for your own case study: full create, read, update and delete (CRUD) on your main entity. This is an **expectation, not a requirement** — the minimum to pass is working `index` and `show` pages for your main entity (the equivalent of Lesson 2).

Deliverables

1. Your **classroom50.org repository**, containing:
   - a history of regular, meaningful commits
   - your Laravel application, which runs in the provided Docker environment
   - migrations, models (with relationships), factories and seeders for your case study
   - controllers and Blade views for your main entity — at minimum `index` and `show`
   - PHPUnit tests that pass
   - a `README.md` describing your case study, listing your entities and their relationships, with a simple entity-relationship diagram (ERD).
2. A **screencast** (maximum 5 minutes), uploaded to your OneDrive folder, in which you:
   - show your application running, including your seeded data

- show `php artisan test` running and passing
- explain your database design: your tables, your 1:M and M:N relationships, and how they appear in your models.

Rubric — Assessment Point 1

Criterion	LOs	Fail (0–39%)	Pass (40–59%)	Merit (60–79%)	Distinction (80–100%)
Data design & models (25%)	LO2, LO3	Migrations are missing, or the 1:M or M:N relationship is missing.	Migrations create all tables, including a 1:M and an M:N relationship with foreign keys. Models define these relationships. A seeder creates some data. README describes the case study.	Factories and seeders produce realistic data for every entity, including the join table. Columns use appropriate data types and constraints. README includes a clear ERD.	The schema enforces the case study's rules in the database (e.g. unique and composite indexes, sensible `on delete` behaviour), and the README explains why.
Application functionality (30%)	LO1, LO2	The application does not run, or the main entity has no working `index` and `show` pages.	Working `index` and `show` pages for the main entity, using route model binding. `show` displays related data from at least one relationship.	Full CRUD on the main entity, with validation, a shared form partial for create and edit, and form repopulation with `old()` after a validation error.	All of Merit, plus work beyond Lesson 3 (e.g. CRUD on a second entity), with clean, consistent, well-organised code.
Testing (15%)	LO2	No tests, or tests do not pass.	Unit tests for at least one model that create a row and check a relationship. All tests pass.	Unit tests for every model, and feature tests for the `index` and `show` pages. All tests pass.	Feature tests for the full CRUD flow, including validation failures. All tests pass.
Screencast (10%)	LO3	No screencast, or it does not show the application.	Shows the application running and tests passing.	Also explains the database design and relationships clearly.	A clear, confident explanation that links what the viewer sees on screen to the code that produces it.
Progress & engagement (20%)	LO1, LO2, LO3	Little or no evidence of regular work. Few commits; poor attendance.	Attends class but shows limited progress. Commits are infrequent or unclear.	Regular commits with clear messages. Attends and works consistently in class.	Consistently works ahead, shows initiative in solving problems, and engages fully in every class.

Assessment Point 2 — Final submission (70%)

Code and screencast due: 23:59, Friday 11 December 2026
Viva: Tuesday 15 December 2026

What we expect

A complete application, covering everything in Lessons 1–6 of the Book Reviews tutorial, plus role-based authorisation, applied to your own case study.

Deliverables

1. Your **classroom50.org repository**, containing:
    - a history of regular, meaningful commits across all 12 weeks
    - your complete Laravel application, including:
        - full CRUD on your main entity
        - CRUD for your other entities, where your case study needs it
        - your M:N relationship managed through your forms (e.g. choosing a book's authors when creating or editing it)
        - search on your main entity
        - **role-based authorisation** — at least two roles, and some pages that can only be accessed by users with a given role. We suggest using **role middleware**, as shown in class. You may use a different approach if you explain and justify it in your design document.
        - any extensions agreed with your lecturer
    - a full PHPUnit test suite that passes
    - your **design document** (see below), as a PDF or Markdown file in a `docs/` folder.
2. A **screencast** (maximum 10 minutes), uploaded to your OneDrive folder, in which you:
    - demonstrate every feature of your application, logged in as each role
    - show `php artisan test` running and passing
    - explain and defend your main design decisions.

The design document

Your design document should include:

1. **Case study** — what your application does and who uses it.
2. **Database design** — an ERD, and an explanation of your tables, relationships and constraints, and why you designed them that way.

3. **Routes** — a table of your routes: HTTP method, URI, controller action, and any middleware.
4. **How a request is handled** — trace one request through your application, from the route to the response, explaining the role of each part of the framework it passes through.
5. **Authorisation** — the roles in your application, what each role can do, how you enforce it, and why you chose that approach.
6. **Testing** — what you test and why, and anything you chose not to test.
7. **Extensions** — what you added beyond the core requirements, and how.
8. **AI Agent Use** — which AI tools you used and what you used them for. Include specific examples: what you asked, what it answered, and what you did with the answer. Reflect critically: when did it help, when did it mislead you, and what did you learn?
9. **Reflection** — what went well, what you would do differently, and any known problems.

Rubric — Assessment Point 2

Criterion	LOs	Fail (0–39%)	Pass (40–59%)	Merit (60–79%)	Distinction (80–100%)
Application functionality (20%)	LO1, LO2	The application does not run, or core functionality is broken.	Full CRUD on the main entity. Users can register and log in.	Also: CRUD for the other entities the case study needs; the M:N relationship is managed through forms; search works; database constraint errors are shown as friendly messages.	All of Merit, plus relationships are eager-loaded (no N+1 queries), error handling is robust, and the code is clean and consistent throughout.
Authentication & authorisation (10%)	LO1, LO2	Little or no authorisation — fewer than two roles, or no pages restricted by role.	At least two roles, and some pages that can only be accessed by users with a given role.	Authorisation checks suit the case study and are applied to every page and action that needs them — e.g. in Book Reviews, only admins can create, edit and delete Books and Authors, and users can only edit and delete their own Reviews.	All of Merit, plus the interface only shows the actions a user is allowed to perform, and the authorisation checks are tested.
Data design (10%)	LO2, LO3	Required relationships are missing or broken.	Schema includes the required 1:M and M:N relationships, with migrations, factories and seeders.	Appropriate data types and constraints. Migrations correct mistakes by adding new migrations, not by editing old ones.	The schema enforces the case study's rules in the database, and the design is clearly justified in the design document.

Criterion	LOs	Fail (0–39%)	Pass (40–59%)	Merit (60–79%)	Distinction (80–100%)
Testing (15%)	LO2	Few or no tests, or tests do not pass.	Tests for every model and for the main entity's CRUD. All tests pass.	Feature tests for every controller, including validation failures, search (matching and non-matching) and authorisation. All tests pass.	A comprehensive, well-organised suite, including tests for what each role can and cannot do, and a test that proves N+1 queries are fixed. All tests pass.
Extensions (10%)	LO2	No agreed extension attempted, or it does not work.	One small extension works.	One substantial extension, or several small ones, working and tested.	Substantial extensions, well integrated, tested, and explained in the design document.
Design document (15%)	LO1, LO3	No design document, or it is missing most sections.	Includes most sections, with limited explanation. AI use is mentioned without detail.	All sections complete, with an ERD and a clear request trace. AI use is described with specific examples.	A clear, professional document that justifies its design decisions, explains the framework's role accurately, and reflects critically on AI use.
Screencast (5%)	LO3	No screencast, or it does not show the application.	Shows the main features and tests passing.	Demonstrates all features and roles, and explains the main design decisions.	A clear, confident demonstration that explains and defends the design.
Progress & engagement (15%)	LO1, LO2, LO3	Little or no evidence of continued work since Assessment Point 1.	Some progress, but commits are infrequent and attendance is inconsistent.	Regular commits throughout, with clear messages. Attends and works consistently in class.	Sustained effort across all 12 weeks, with initiative shown beyond the core requirements.

Submission checklist

Before each deadline, check that:

- [ ] All your work is committed **and pushed** to your classroom50.org repository.
- [ ] `php artisan migrate:fresh --seed` runs without errors.
- [ ] `php artisan test` runs, and all tests pass.
- [ ] Your screencast is uploaded to your OneDrive folder and is within the time limit.
- [ ] (Assessment Point 1) Your `README.md` describes your case study and includes an ERD.
- [ ] (Assessment Point 2) Your design document is in the `docs/` folder.